



Parameterized Graph Algorithms and Their Role in Static Program Analysis

GIOVANNA KOBUS CONRADO

Thesis Defense

Advised by: Amir Goharshady and Pedro Sander

Contents

01. Introduction

02. Parameterization

03. Graphs in Static Analysis

→ **Control-Flow Graphs**

→ **Call Graphs**

→ **Data-Flow Graphs**

Introduction

```

int labelid[112345];

string s;

vector<pii> are[112345];

char type[112345];
int cid=1;

pair<pii,int> prop(int i){
    if(i>=s.size())return mp(-1,-1),-1);
    if(s[i]==' '){
        pii sla=prop(i+1);
        if(sla.f.f!=-1)sla.f.f=1;
        return sla;
    }
    if(s[i]=='')return mp(-1,-1),i+1);
    int mt=cid;
    cid++;
    type[mt]=s[i];
    int nxt=i+1;
    while(s[nxt]>='0' && s[nxt]<='9'){
        if(labelid[s[nxt]-'0']==0)
            labelid[s[nxt]-'0']=mt;
        else{
            are[mt].pb(mp(0,labelid[s[nxt]-'0']));
            are[labelid[s[nxt]-'0']].pb(mp(0,mt));
            labelid[s[nxt]-'0']=0;
        }
        nxt++;
    }
    while(s[nxt]=='('){
        pii sla=prop(nxt+1);
        nxt=sla.s;
        if(sla.f.f!=-1){
            are[mt].pb(sla.f);
            are[sla.f.s].pb(mp(sla.f.f,mt));
        }
    }
    pii sla=prop(nxt);
}

```

How do we know if a program works?

We can run it...

Can we catch mistakes **before they happen?**

What is static analysis?

Analysis of program behaviour **before we run it**

- **Code optimization** (how can I use less memory?)
- **Security** (can a user access private data?)
- **Correctness** (does this function do what I want?)

These must reason about for **all possible executions**

What is static analysis?

How do we even reason about programs?

Program ☐ **Graph**

Static analysis problem ☐ **Graph problem**

- Most of these problems are hard or undecidable
- Programs are famously hard to reason about

Have we lost all hope? Well...

Hardness results are a worst case scenario.

What if programs are all good cases?

Have we lost all hope? Well...

Hardness results are a worst case scenario.
What if programs are all good cases?

We can prove that is often true

How?

Parameterization

Parameterization allows us to describe graphs that come from programs, in this way we can:

- Formally describe properties of programs
- Prove them, or test them
- Write algorithms that are provably efficient and provably correct

This is what this thesis is about

Parameterization

Parameterized algorithms

Analysing algorithms with respect to multiple parameters

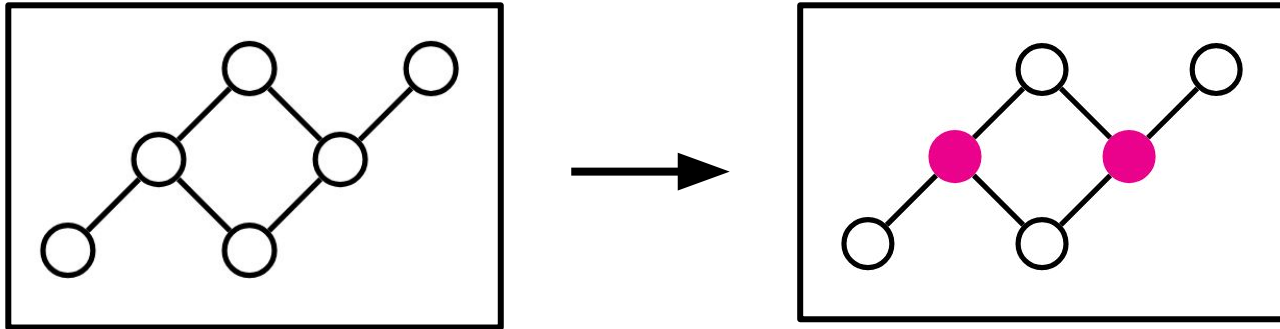
$$O(n) \rightarrow O(n \cdot k)$$

k can be:

- **A limit on the output** (minimum → is it less than k ?)
- **A limit on the input** (graphs with degree at most k)
- **Something fun** (dimensions, factor for approximation)

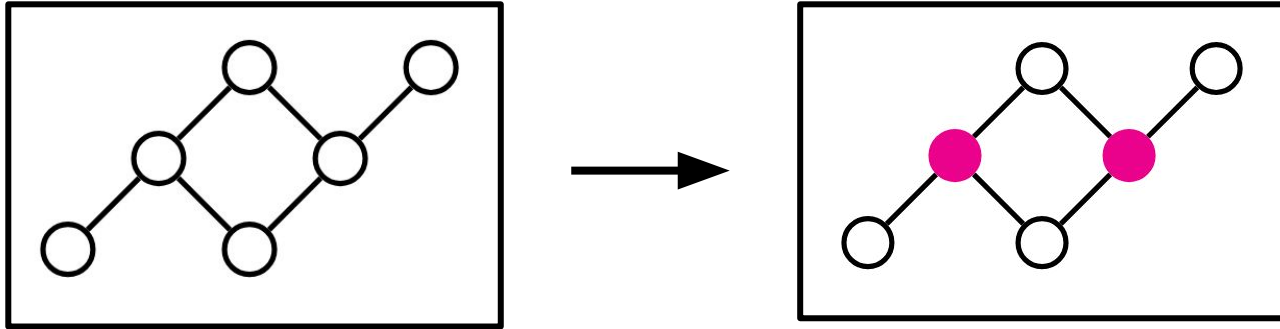
Vertex Cover

Minimum $S \subseteq V$ such that $G[V \setminus S]$ has no edges



Vertex Cover

Minimum $S \subseteq V$ such that $G[V \setminus S]$ has no edges

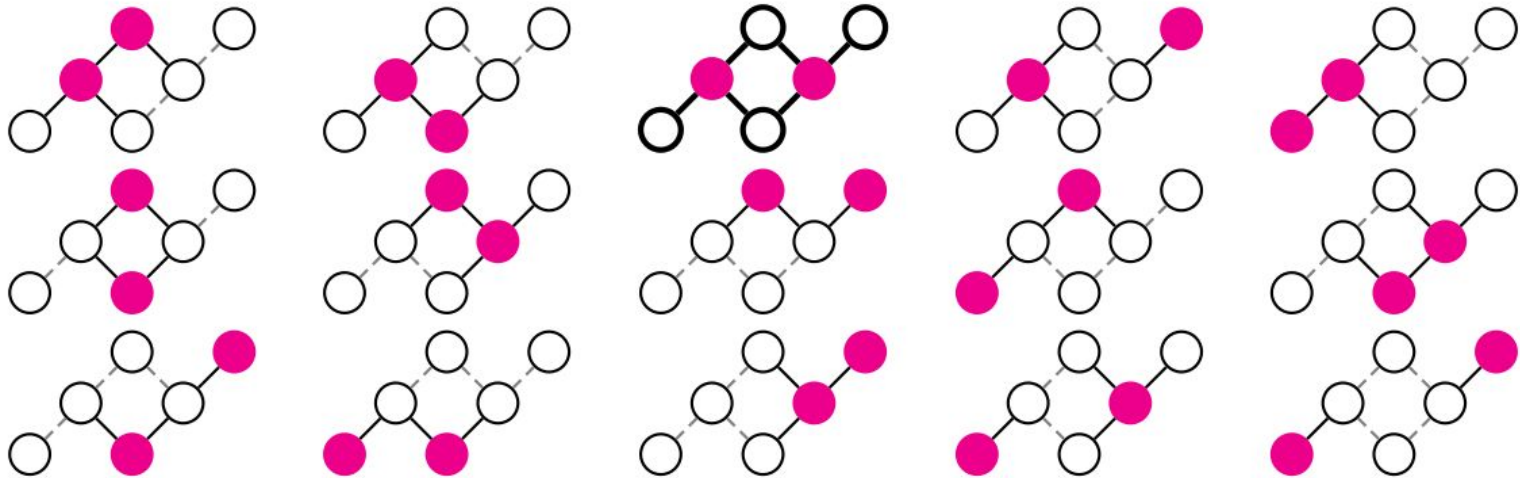


NP-Complete

What if we only want to know if the vertex cover is smaller than **k**?

Vertex Cover

If k is small we can test all sets of k vertices and check



Vertex Cover

What is the complexity of this?

$$O\left(\binom{|V|}{k} \cdot |E|\right) \subseteq O(|V|^k \cdot |E|)$$

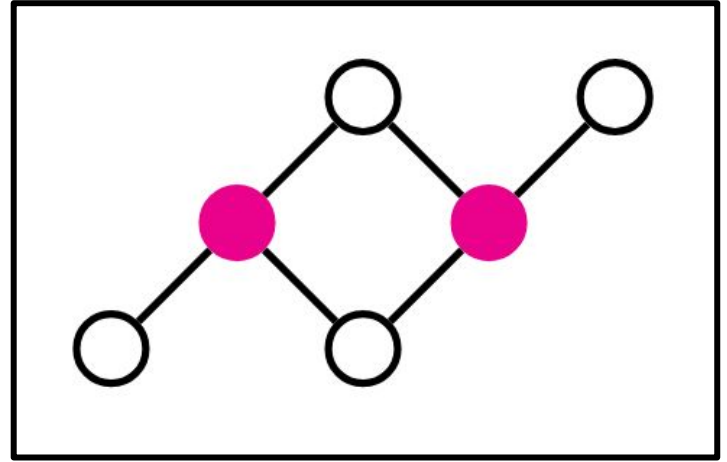
$$\rightarrow O(n^{f(k)})$$

Complexity class is piecewise polynomial (XP)

Vertex Cover

We can do better

All edges are adjacent to a vertex in the cover $\square$ For any edge, one of its endpoints is in the cover



Vertex Cover

Consider now the following algorithm:

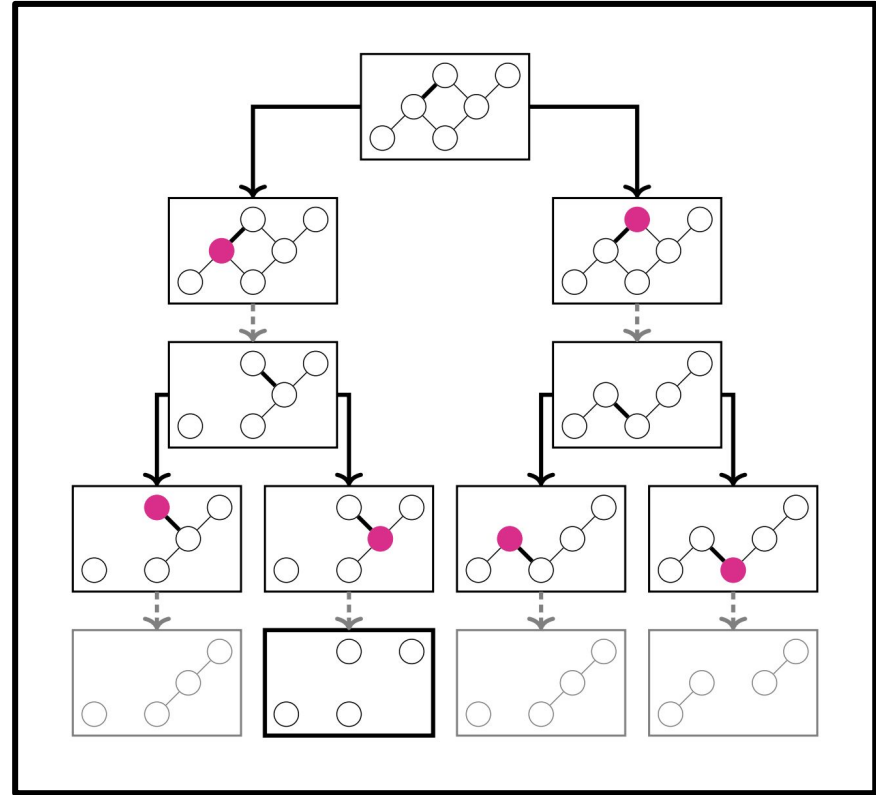
1. Receive **(G, k)**
2. If G has no edges:
 - Return **true**
3. If k is zero:
 - Return **false**
4. Take an edge $e \in E$:
 - For each endpoint v of e :
 - Solve for **(G \setminus v, k-1)**

Since every edge has to be covered, this algorithm always finds a vertex cover of size **k** if there is one

Vertex Cover

Recursion tree:

- **2** children per node
- Depth **k**
- **2^k** leaves



Vertex Cover

What is the complexity of exploring the recursion tree?

$$O(2^k(|V| + |E|))$$

$$\rightarrow O(f(k) \cdot n^c)$$

Complexity class is **fixed-parameter tractable (FPT)**

In static analysis, we aim for linear FPT algorithms

$$\rightarrow O(f(k) \cdot n)$$

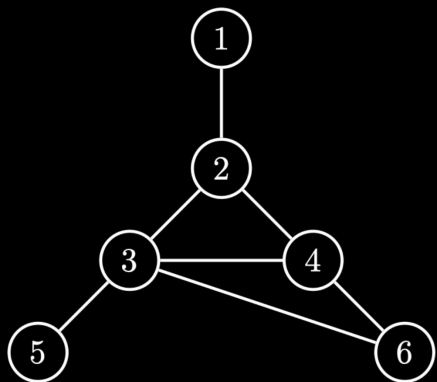
Structural parameters

Generally hard graph problems are easy for some graph classes.

- Trees
- Parallel-series graphs
- Planar graphs

Somehow, these graphs are “simpler” than others.

How do we capture simplicity in a graph?



Treewidth

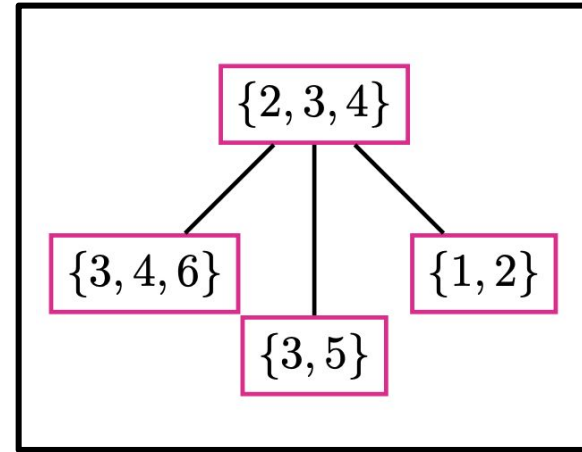
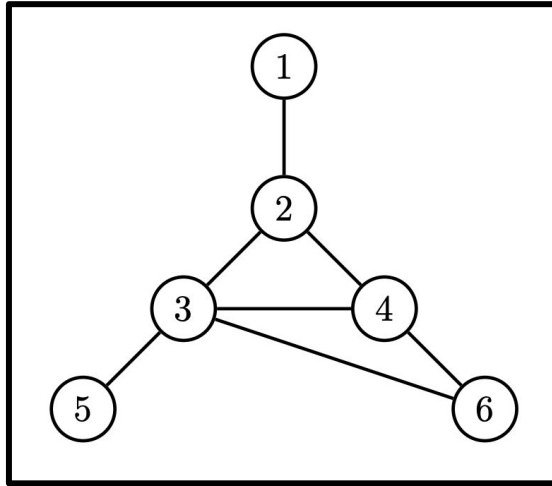
How similar is this graph to a tree?

A graph has treewidth **k** if it has a tree decomposition of width **k**

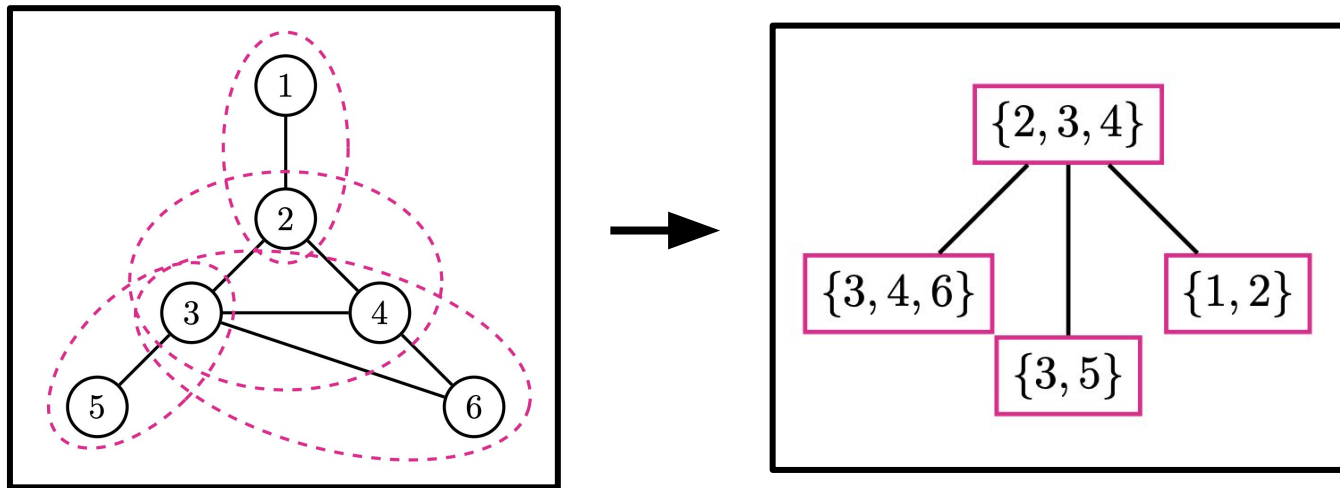
Many “simple” graphs admit low-width tree decompositions

Tree decomposition:

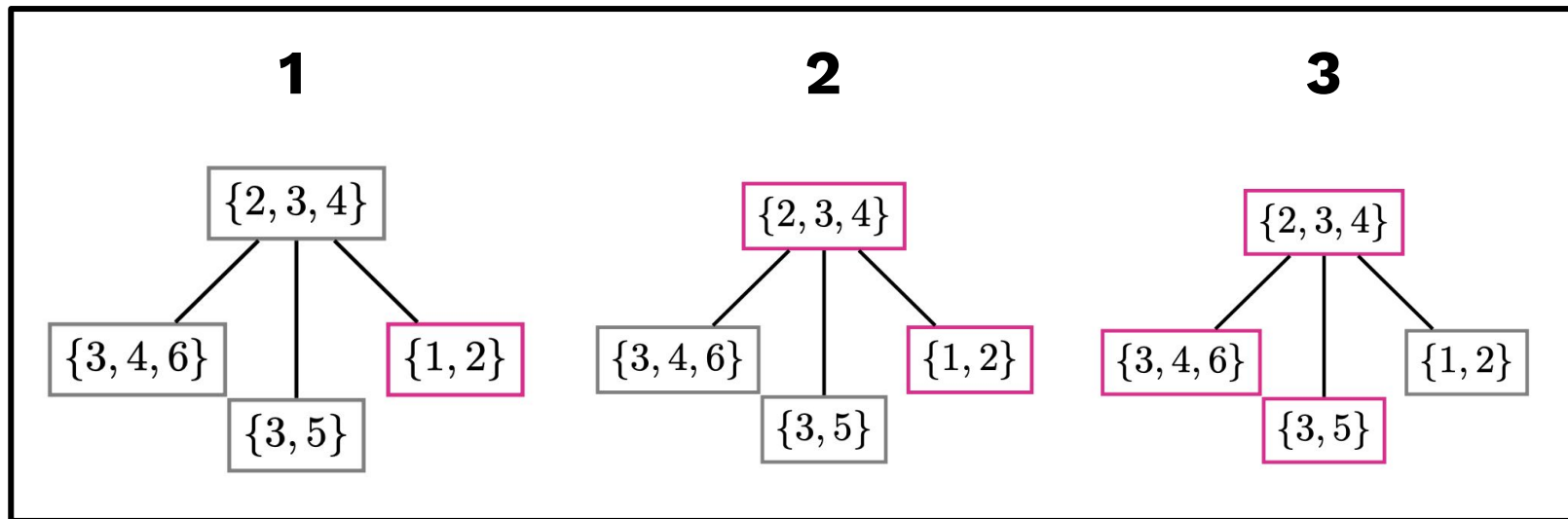
Tree of subsets of V , $\mathcal{T} = (T, \{X_t\}_{t \in V(T)})$, such that:



For every $uv \in E(G)$, there exists a node $t \in V(T)$ such that $\{u, v\} \subseteq X_t$.

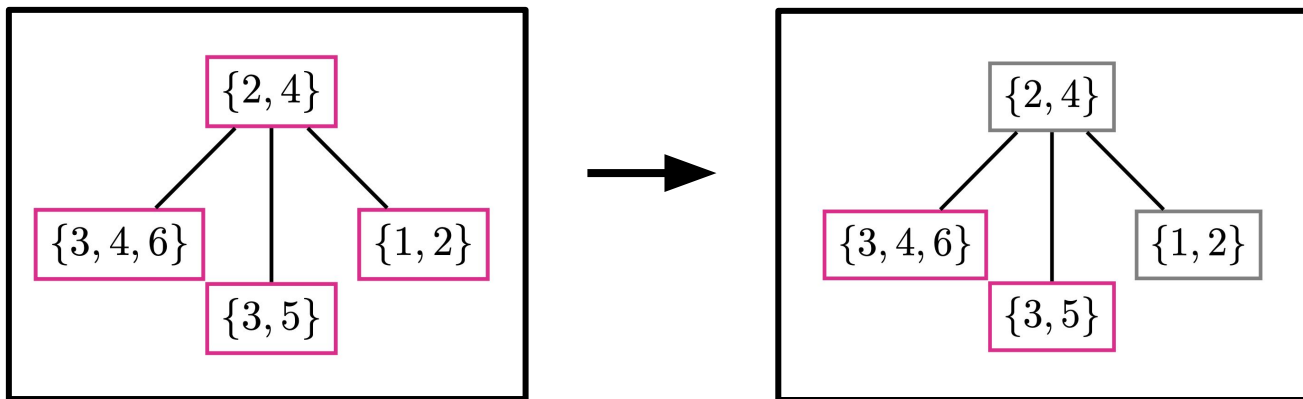


For every $v \in V(G)$, the set $T_v := \{t \in V(T) : v \in X_t\}$, induced graph $T[T_v]$ is nonempty subtree of the T .



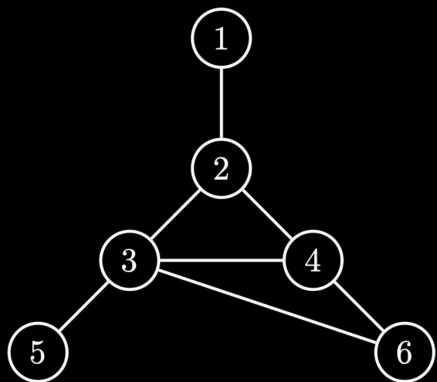
For every $v \in V(G)$, the set $T_v := \{t \in V(T) : v \in X_t\}$, induced graph $T[T_v]$ is nonempty subtree of the T .

not allowed



The width of a tree decomposition is the size of its largest bag minus one

The **treewidth** of a graph is the minimum width of a tree decomposition of that graph



Pathwidth

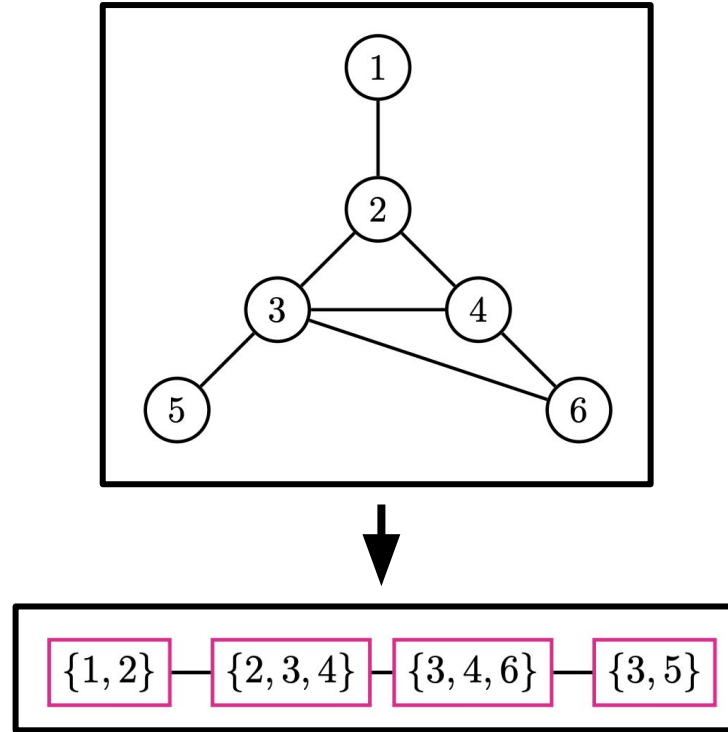
How similar is this graph to a path?

A graph has pathwidth **k** if it has a path decomposition of width **k**

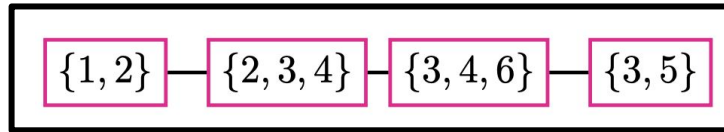
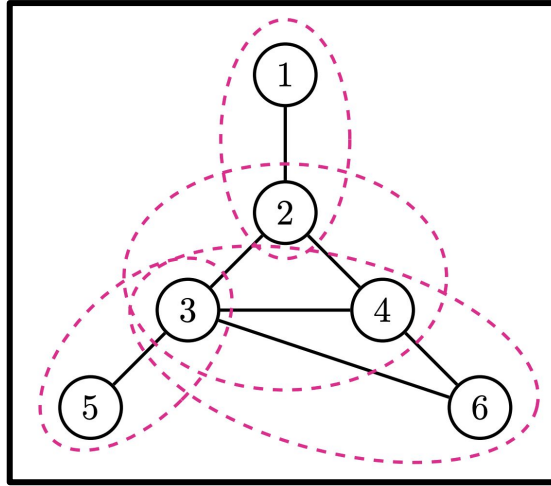
Paths $\subset$ Trees

Path decompositions $\subset$ Tree decompositions

Sequence of subsets of V , $\mathcal{P} = \{X_1, \dots, X_r\}$, such that:



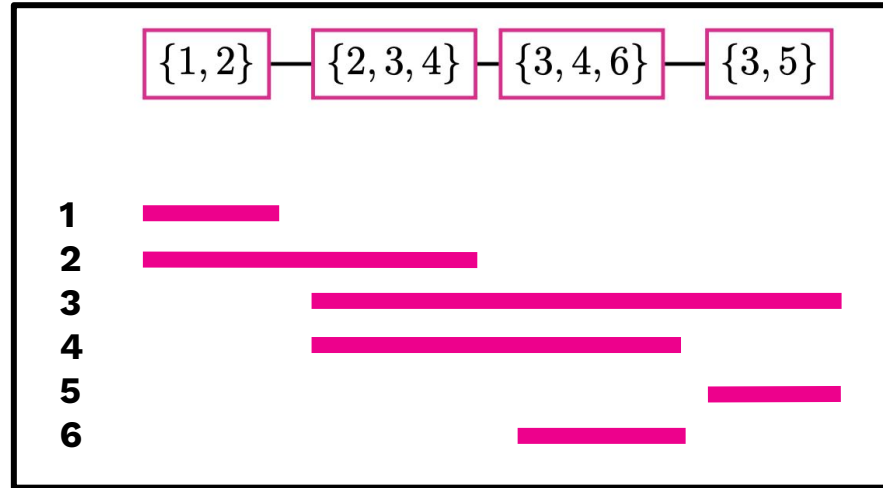
For each $uv \in E$, there exists an i such that $\{u, v\} \subseteq X_i$.



For each $v \in V$, there exists a pair of indices $1 \leq l(v) \leq r(v) \leq r$ such that

$$v \in X_i \iff l(v) \leq i \leq r(v).$$

In other words each graph vertex maps to continuous subpath of the decomposition.



The width of a path decomposition is the size of its largest bag minus one

The **pathwidth** of a graph is the minimum width of a path decomposition of that graph

Pathwidth is stronger than treewidth: $\text{pw}(G) \geq \text{tw}(G)$

Dynamic programming on path-/tree-decompositions

- Dynamic programming does not work on cycles
- But it works on paths (arrays) and trees

Can we also use dynamic programming on tree-like graphs?

- Common $O(n)$ technique for NP-Hard problems
- Similar for pathwidth

Computing structural parameters

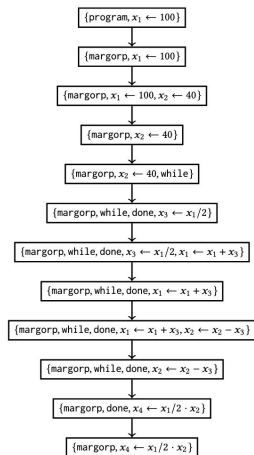
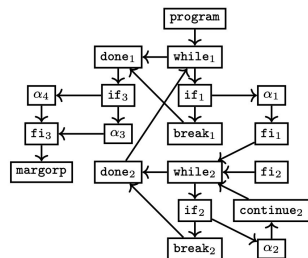
How do I find the treewidth of a graph?

- FPT with respect to the parameter
- Not practical
- Case-by-case basis
- Heuristics

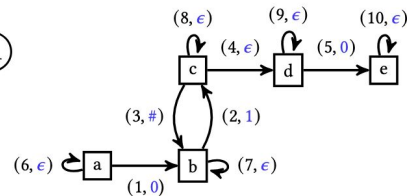
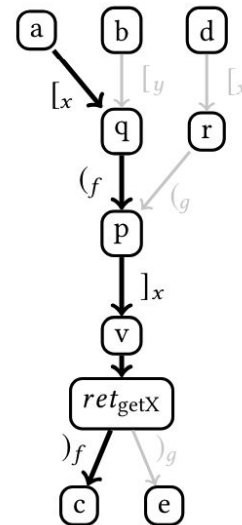
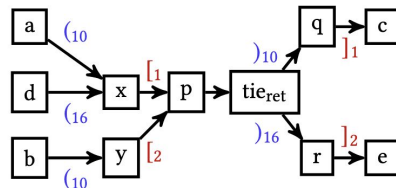
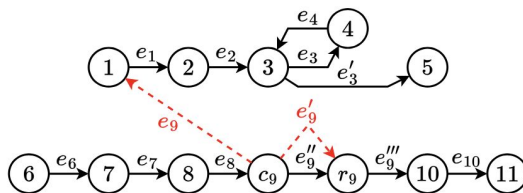
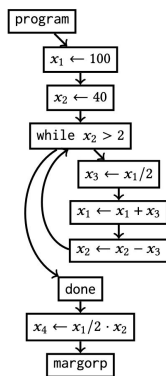
Computing treewidth* : $O(n)2^{O(k^3)}$

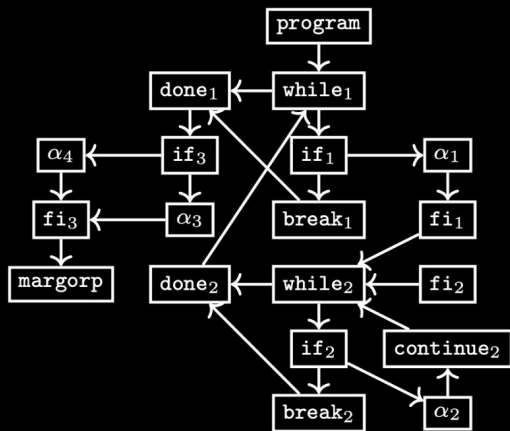
(*) A linear time algorithm for finding tree-decompositions of small treewidth,
H. Bodlaender, 1996

Graphs in Static Analysis



There are many graphs!





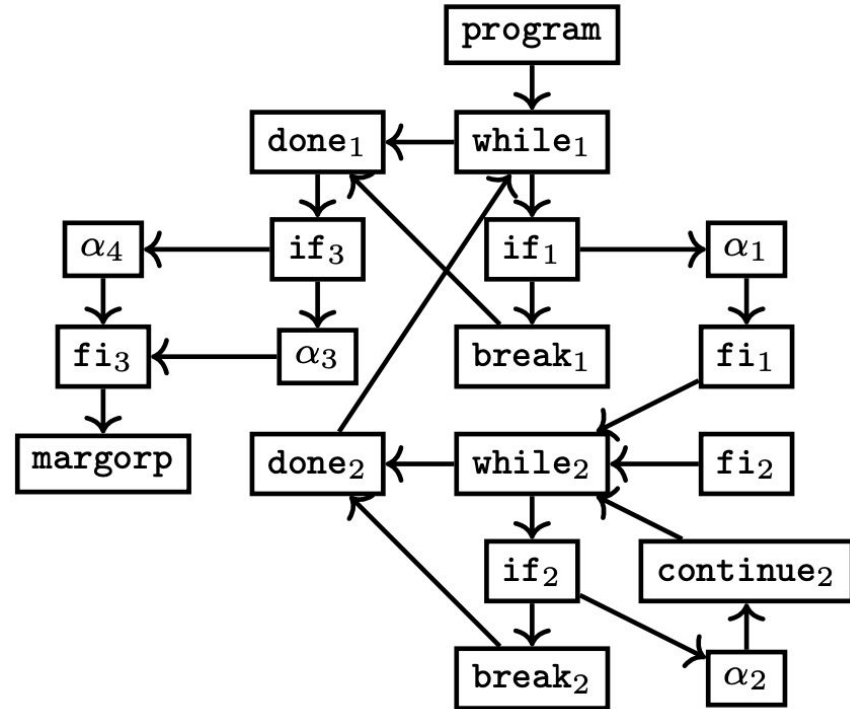
Control-Flow Graphs

- Represents the execution paths on a function
- Used in intraprocedural program analysis

```

program
  while1 B do
    if1 B then
      break1
    else
      α1
      fi1;
  while2 B do
    if2 B then
      break2
    else
      α2;
      continue2
    fi2
  done2
done1;
if3 B then
  α3
else
  α4
fi3
margorp

```



Bound on control-flow graphs

Control-flow graphs tend to have a specific shape

- All control-flow graphs of structured programs have a treewidth of at most 7

[G.K. Conrado](#), A.K. Goharshady, C.K. Lam

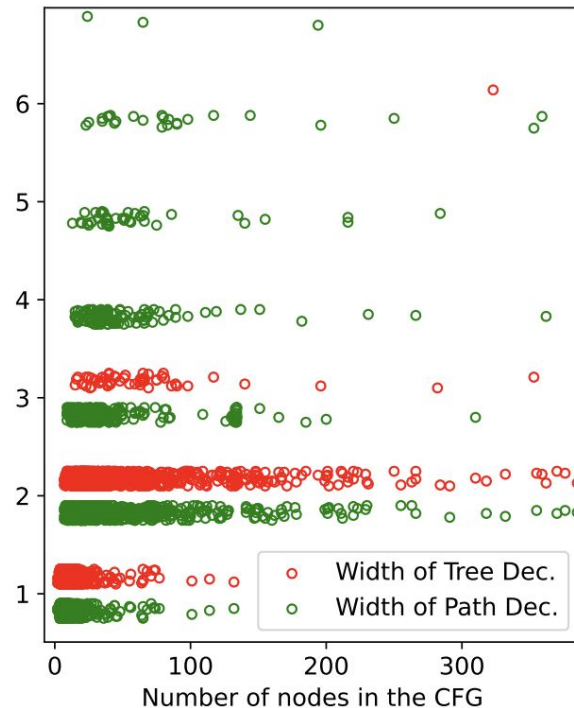
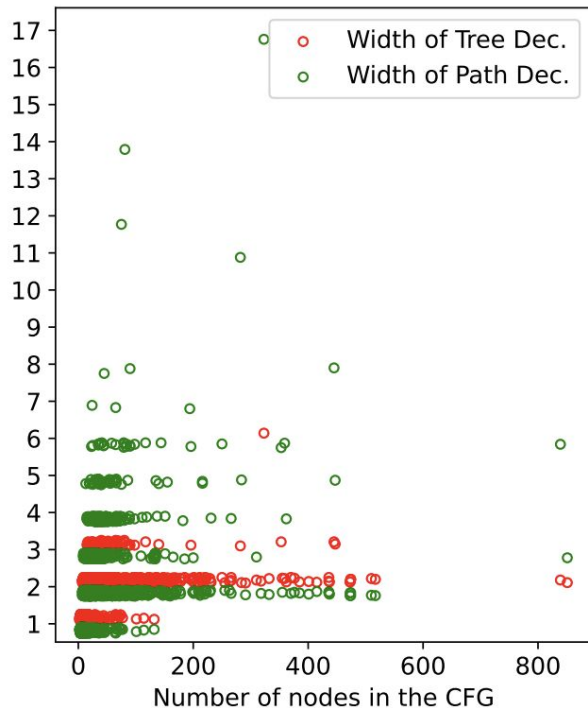
The Bounded Pathwidth of Control-Flow Graphs

OOPSLA 2023

- All control-flow graphs of structured programs have a pathwidth of at most twice their nesting depth

Bound on control-flow graphs

Width	Instances	
	TD	PD
1	3276	3276
2	1228	849
3	55	238
4	0	119
5	0	42
6	1	26
7	0	3
8	0	3
11	0	1
12	0	1
14	0	1
17	0	1



Register allocation

Map variables to registers

Variables being used at the same time are assigned different registers

We want to find **if we can map variables to r registers without spills**



Register Allocation

Graph Coloring on CFG

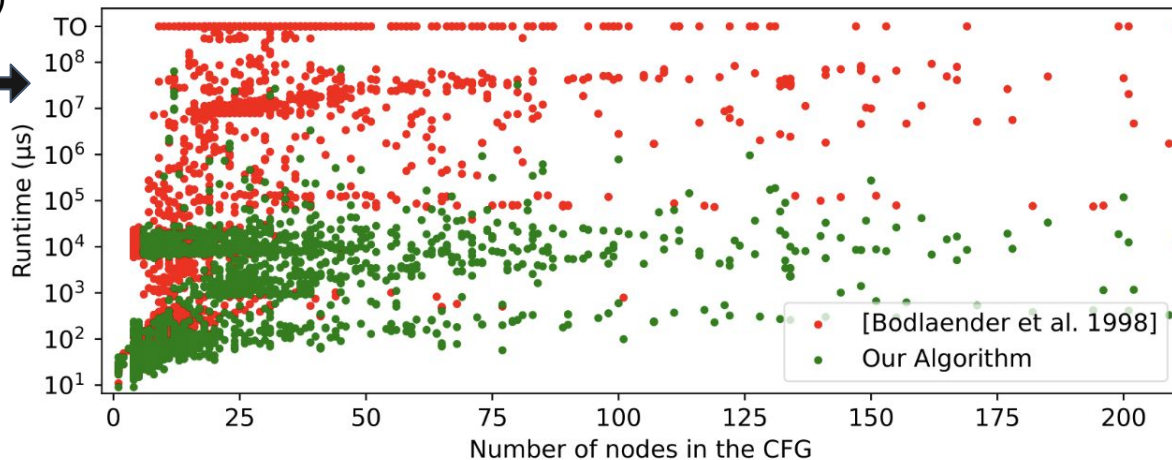
DP on Path-/Tree-Decomposition

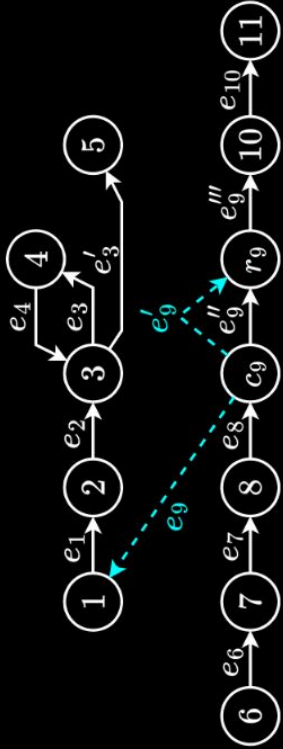
Register allocation

We solve using DP on the tree-/path-decomposition

DP on paths is simpler than DP on trees

(logarithmic)



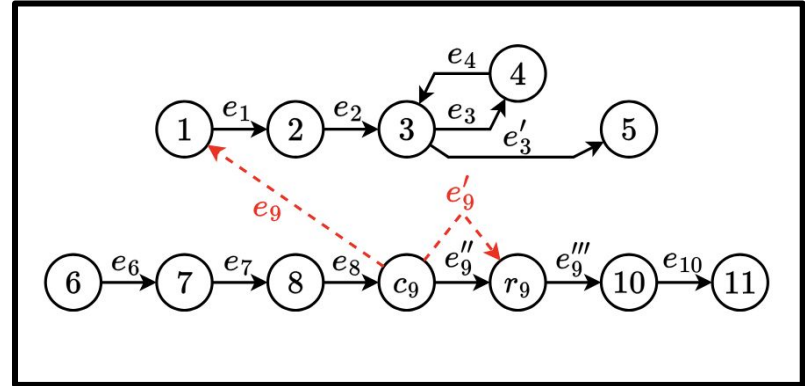


Call Graphs

- Represents the function calls of a program
- Used in interprocedural program analysis

Call graphs work alongside control-flow graphs to represent the full program

```
1 void f(int *p, int *&q){
2   unsigned int temp; cin>>temp;
3   while(temp-->0)
4     swap(p, q);
5 }
6 int main(){
7   int *p, *q;
8   p = new int(100);
9   f(p, q);
10  return *q;
11 }
```



Call graphs can have arbitrary shape

$f_1() \{ \}$	$f_2() \{$ $f_1();$ $\}$	$f_3() \{$ $f_1();$ $f_2();$ $\}$	$f_4() \{$ $f_1();$ $f_2();$ $f_3();$ $\}$
---------------	--------------------------------	--	--

Bound on call graphs

- Experimentally, call graphs have bounded treewidth
- They also have bounded treedepth, a stronger parameter

		treedepth ↓	
	$ \widehat{G} $	tw	td
hsqldb	1881	3	4
xalan	2204	3	4
avroa	4632	3	13
fop	9473	4	11
luindex	22207	4	15
lusearch	29380	4	14
eclipse	38500	5	27
antlr	43485	4	44
pmd	76578	4	50
sunflow	105510	4	100
jython	110446	6	65
chart	122517	7	63
bloat	128501	7	133
Average	53486	4.46	41.77

Bound on call graphs

[G.K. Conrado](#), A.K. Goharshady, K. Kochekov, Y.C. Tsai, A.K. Zaher

**Exploiting the Sparseness of Control-flow and Call Graphs
for Efficient and On-demand Algebraic Program Analysis**

OOPSLA 2023

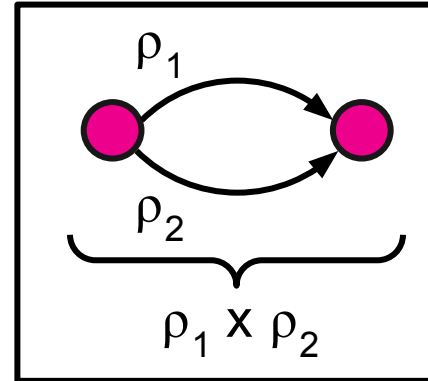
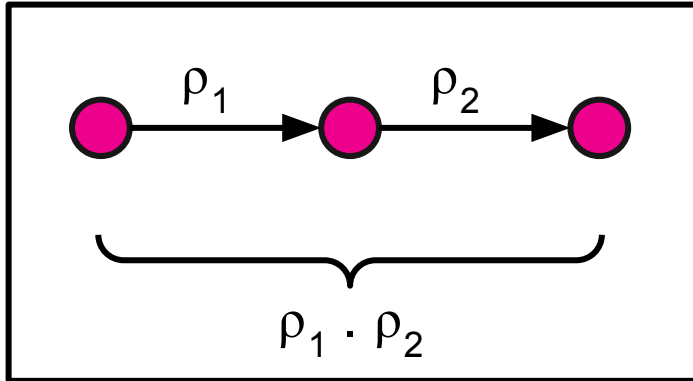
Algebraic Program Analysis

We represent our property as an **algebra**

$$\rho ::= e \mid \emptyset \mid \epsilon \mid \rho + \rho \mid \rho \cdot \rho \mid \rho^* \quad e \in E$$

Element are summary of program behaviour

Operations define how to compose elements



Algebraic Program Analysis

We query the summary of the paths between two points of a program

→ Using the structural properties, we answer on-demand queries in $O(t)$, with $O(n \cdot \log n \cdot t)$ preprocessing

t is the time needed to perform a single atomic operation in the algebra.

This result does not rely on dynamic programming

We use the structural bounds to decompose the program graph

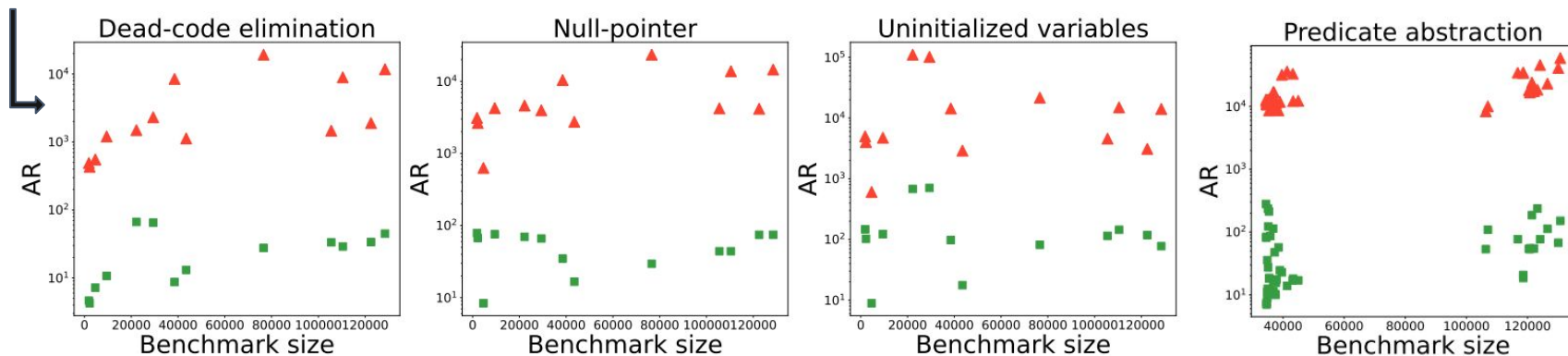
Algebraic Program Analysis

In our experiments we performed two types of analyses:

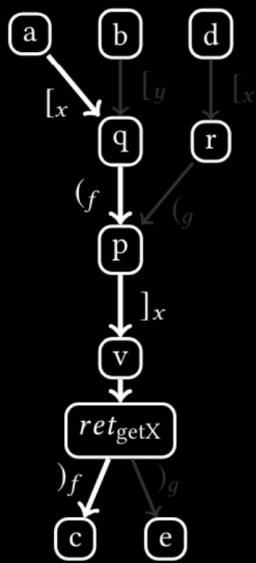
- **Data-flow analysis:** dead-code elimination, possibly-uninitialized variables, null-pointer analysis
- **Predicate abstraction:** boolean program analysis

Algebraic Program Analysis

(logarithmic)



our algorithm vs **baseline**



Data-flow graphs

- Represents the flow of data in a program through variable assignments
- We do not rely on structural bounds

Problem

Can the value of variable **a** flow into variable **c**?

```
int  a ← 2
```

```
int  b ← a
```

```
int  c ← b
```

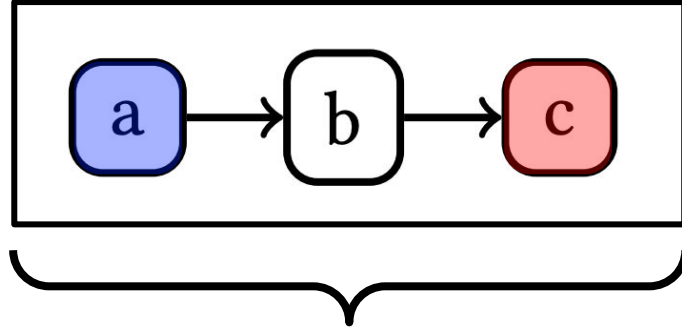
One solution

Can the value of variable **a** flow into variable **c**?

```
int a ← 2
```

```
int b ← a
```

```
int c ← b
```



Check if **a** reaches **c**

What if there are function calls?

Which variables affect the value of **b**?

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```

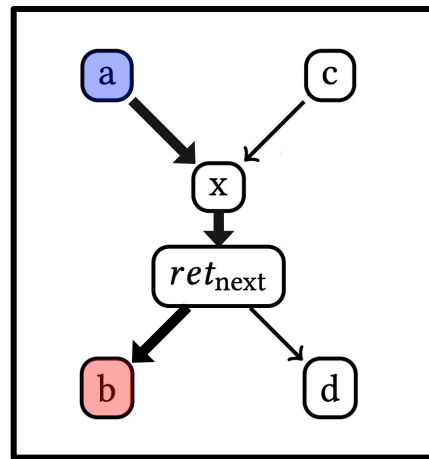
What if there are function calls?

Which variables affect the value of **b**?

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```



a reaches **b**...

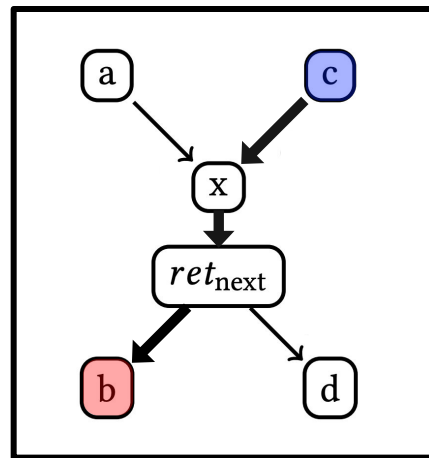
What if there are function calls?

Which variables affect the value of **b**?

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```



a reaches **b**... but so does **c**

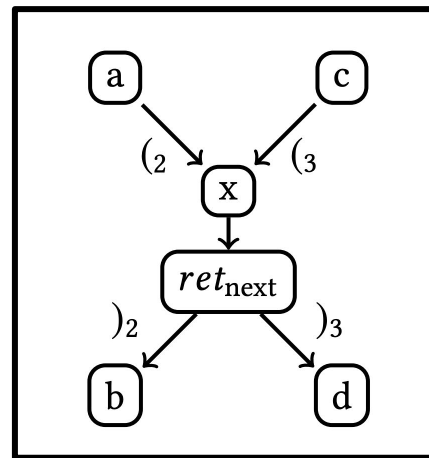
Context-sensitivity

We use parentheses to model calling contexts

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```



the path must be well-nested

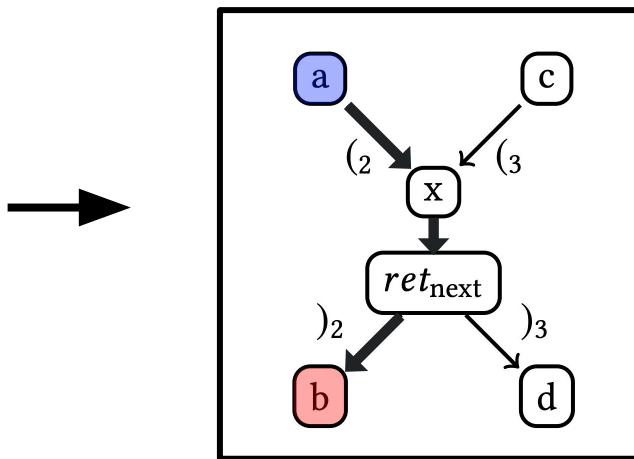
Context-sensitivity

We use parentheses to model calling contexts

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```



a to **b** is context-sensitive $\rightarrow$ word $(_2)_2$

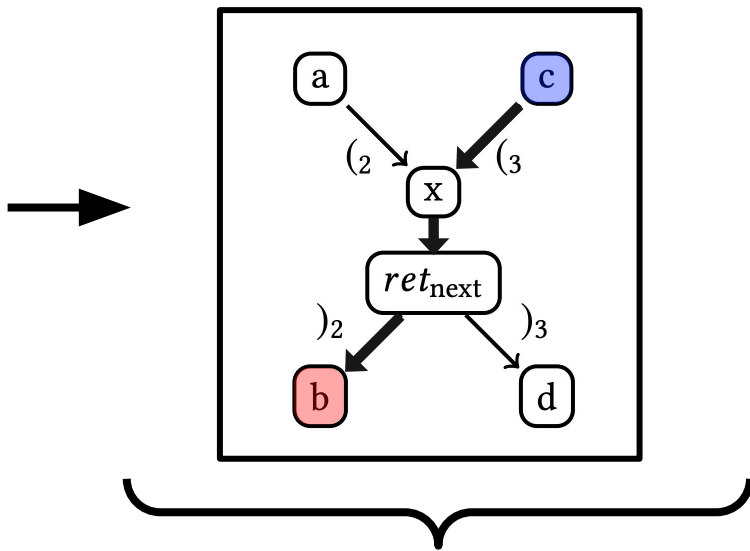
Context-sensitivity

We use parentheses to model calling contexts

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```

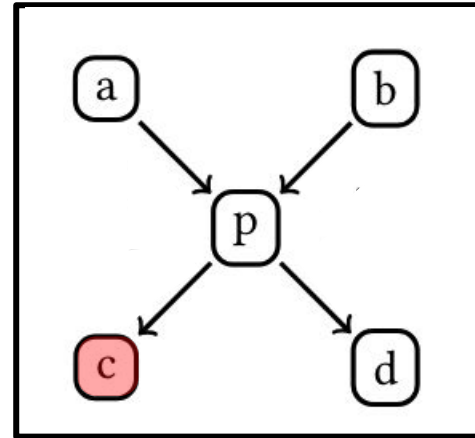


c to **b** is not context-sensitive $\rightarrow$ word $(_3)_2$

What if there are composite data types?

Which variables affect the value of **c**?

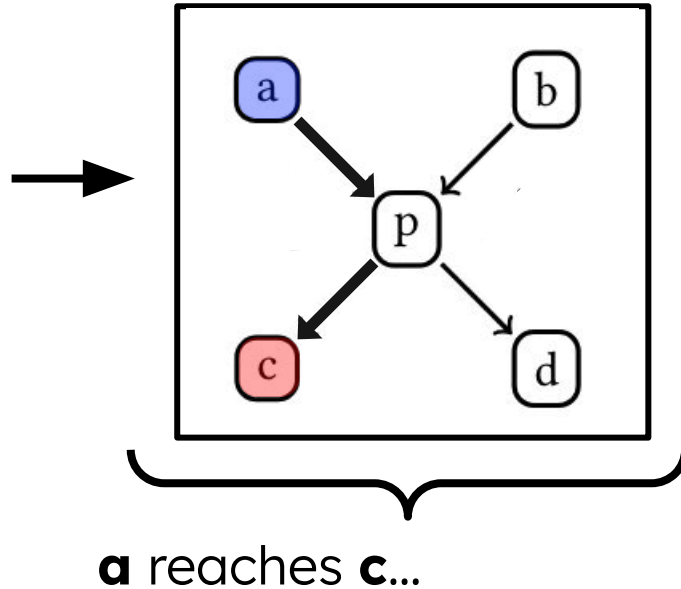
```
int a ← 2  
int b ← 3  
point p  
p.x ← a  
p.y ← b  
int c ← p.x  
int d ← p.y
```



What if there are composite data types?

Which variables affect the value of **c**?

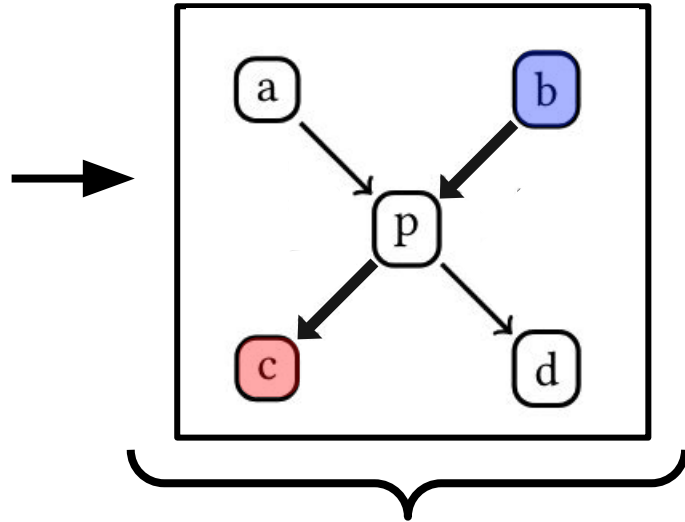
```
int a ← 2  
int b ← 3  
point p  
p.x ← a  
p.y ← b  
int c ← p.x  
int d ← p.y
```



What if there are composite data types?

Which variables affect the value of **c**?

```
int a ← 2  
int b ← 3  
point p  
p.x ← a  
p.y ← b  
int c ← p.x  
int d ← p.y
```

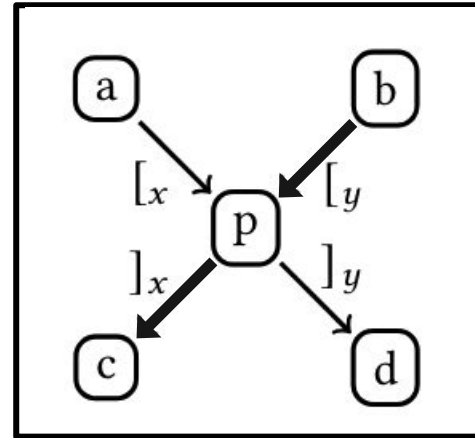


a reaches **c**... but so does **b**

Field-sensitivity

We use brackets to model fields

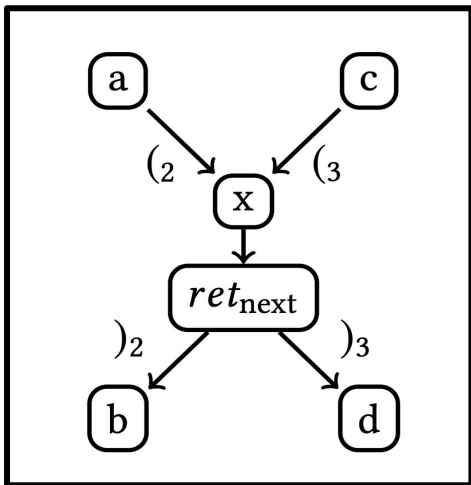
```
int  a ← 2  
int  b ← 3  
point p  
p.x ← a  
p.y ← b  
int  c ← p.x  
int  d ← p.y
```



Language reachability

Given a graph **G**, which pairs of vertices can reach other through a path in the language **L**

Widely used framework for static analysis

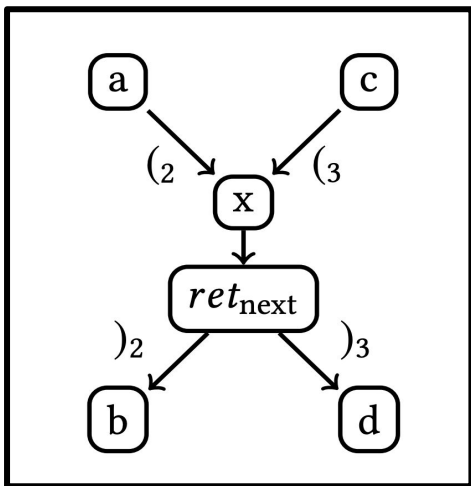


- **G** is the object graph
- **L** is the language of well-nested parentheses: **Dyck language**

Language reachability

Given a graph **G**, which pairs of vertices can reach other through a path in the language **L**

Widely used framework for static analysis



- G is the object graph
- L is the language of well-nested parentheses: **Dyck language**

this is a context-free language (**CFL**)

CFL reachability

CFLs are languages that can be represented by context-free grammars

Language reachability when the language is a CFL can be solved in **$O(n^3)$ time**

(good)

CFL reachability

CFLs are languages that can be represented by context-free grammars

Language reachability when the language is a CFL can be solved in **$O(n^3)$ time**

(good)

What about when it isn't?

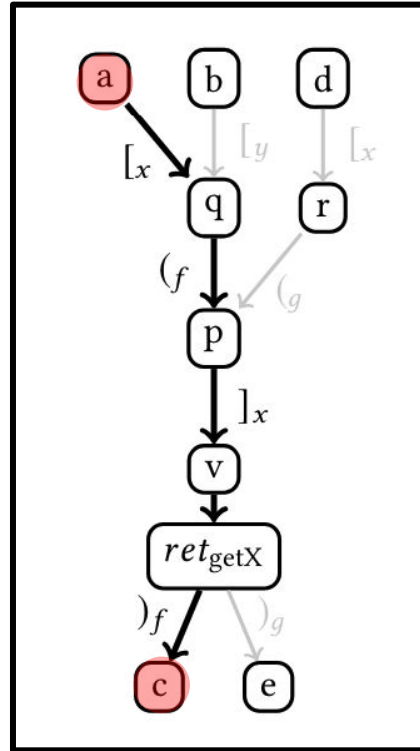
What about when we are dealing with multiple languages at once?

Context and field-sensitivity, simultaneously

```
fun getX(p)
  int v ← p.x
  return v

fun f()
  int a ← 2
  int b ← 3
  point q
  q.x ← a
  q.y ← b
  int c ← getX(q)

fun g()
  int d ← 1
  point r
  r.x ← d
  int e ← getX(r)
```

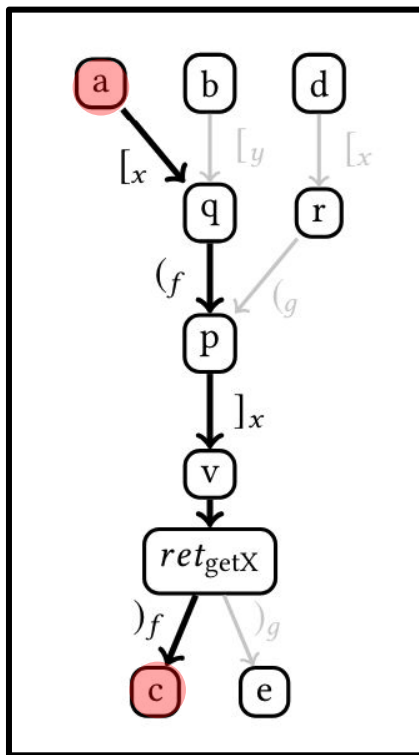


Context and field-sensitivity, simultaneously

```
fun getX(p)
  int v ← p.x
  return v

fun f()
  int a ← 2
  int b ← 3
  point q
  q.x ← a
  q.y ← b
  int c ← getX(q)

fun g()
  int d ← 1
  point r
  r.x ← d
  int e ← getX(r)
```



Parenthesis and brackets may be **interleaved**

$[_x ({}_f]_x)_f$

$[_x ({}_f]_x)_f$

$[_x ({}_f]_x)_f$

Context and field-sensitivity, simultaneously

This is called the Interleaved Dyck Language

- **IDL reachability is undecidable***

Therefore we must approximate it

→ I worked on **underapproximations** and **overapproximations**

(*) Undecidability of context-sensitive data-dependence analysis, **T. Reps**, 2000

Interleaved Dyck Reachability

Overapproximation

[G.K. Conrado](#), A. Pavlogiannis

A Better Approximation for Interleaved Dyck Reachability

SOAP 2024

[G.K. Conrado](#), A. Pavlogiannis

CFL-based Methods for Approximating Interleaved Dyck Reachability

STTT 2025

Context-sensitive Languages

Context-free Languages

Dyck language English grammar*

Palindromes

$b^n a^{n+m} b^m$

Dutch grammar*

$s_1 s_2 s_2 s_1$

Word repeated twice

interleaved Dyck language

a^p , p is prime

*at least

Context-sensitive Languages

Context-free Languages

Dyck language

English grammar*

Palindromes

$b^n a^{n+m} b^m$

Dutch grammar*

$s_1 s_2 s_2 s_1$

Word repeated twice

interleaved Dyck language

a^p , p is prime

*at least

Interleaved Dyck Reachability

We developed **increasingly precise** CFLs that overapproximate the IDL and combined it previous methods*

Reduced reported pairs by half

Potential false positive rate from 55% to 12%

(*) Mutual Refinements of Context-Free Language Reachability,
Ding and Qirun Zhang, 2023

Interleaved Dyck Reachability

Underapproximation

[G.K. Conrado](#), A.H. Kjelstrøm, A. Pavlogiannis, J. van de Pol

Program Analysis via Multiple Context Free Language Reachability

POPL 2025

Context-sensitive Languages

Multiple Context-free Languages

Context-free Languages

Dyck language English grammar*

Palindromes

$b^n a^{n+m} b^m$

Dutch grammar*

$s_1 s_2 s_2 s_1$

Word repeated twice

interleaved Dyck language

a^p , p is prime

→ We approximate IDL through an MCFL because MCFL-reachability is decidable

*at least

d-MCFL(r)

→ Rules and predicates

Language

$$\{w_1 w_2 \# w_2 w_1 \mid w_1, w_2 \in \{0, 1\}^*\}$$


Grammar

$$A(\epsilon, \epsilon)$$
$$A(x_1 0, x_2 0) \leftarrow A(x_1, x_2)$$
$$A(x_1 1, x_2 1) \leftarrow A(x_1, x_2)$$
$$\mathcal{S}(x_1 y_1 \# y_2 x_2) \leftarrow A(x_1, x_2), A(y_1, y_2)$$

d-MCFL(r)

- Rules and predicates
- **Dimension d**

A produces **two** strings simultaneously, so **d=2**

Language

$$\{w_1 w_2 \# w_2 w_1 \mid w_1, w_2 \in \{0, 1\}^*\}$$


Grammar

$A(\epsilon, \epsilon)$

$$A(x_1 0, x_2 0) \leftarrow A(x_1, x_2)$$
$$A(x_1 1, x_2 1) \leftarrow A(x_1, x_2)$$
$$S(x_1 y_1 \# y_2 x_2) \leftarrow A(x_1, x_2), A(y_1, y_2)$$

d-MCFL(r)

- Rules and predicates
- **Dimension d**
- **Rank r**

A produces **two** strings simultaneously, so **d=2**

There are **two** predicates on the RHS, so **r=2**

Language

$$\{w_1 w_2 \# w_2 w_1 \mid w_1, w_2 \in \{0, 1\}^*\}$$



Grammar

$$A(\epsilon, \epsilon)$$

$$A(x_1 0, x_2 0) \leftarrow A(x_1, x_2)$$

$$A(x_1 1, x_2 1) \leftarrow A(x_1, x_2)$$

$$\mathcal{S}(x_1 y_1 \# y_2 x_2) \leftarrow A(x_1, x_2), A(y_1, y_2)$$

d-MCFL(r)

- Rules and predicates
- **Dimension d**
- **Rank r**
- For $d=1$, it is a CFL

A produces **two** strings simultaneously, so **d=2**

There are **two** predicates on the RHS, so **r=2**

Language

$$\{w_1 w_2 \# w_2 w_1 \mid w_1, w_2 \in \{0, 1\}^*\}$$



Grammar

$$A(\epsilon, \epsilon)$$

$$A(x_1 0, x_2 0) \leftarrow A(x_1, x_2)$$

$$A(x_1 1, x_2 1) \leftarrow A(x_1, x_2)$$

$$\mathcal{S}(x_1 y_1 \# y_2 x_2) \leftarrow A(x_1, x_2), A(y_1, y_2)$$

Approximating IDL

([)], (([)], ()[], [([([)])]]]

Strings in IDL (labels omitted)

Can we underapproximate this using a CFL?

Approximating IDL as MCFL

Idea: use dimensions as "breaking points" for interleaving

$P^3(((,))(,))$

$Q^3([[,]][[,])$

Approximating IDL as MCFL

$$\mathcal{S}(x_1y_1x_2y_2x_3y_3) \leftarrow P^3(x_1, x_2, x_3), Q^3(y_1, y_2, y_3)$$

Diagram illustrating the mapping from P^3 and Q^3 to S . The top row shows $P^3(((,)) (,))$ and $Q^3([[,]] [,])$. The bottom row shows $S((([[)) (]] [)])$. Arrows indicate the mapping: the first two magenta arcs of P^3 map to the first two magenta arcs of S , and the third magenta arc maps to the fourth magenta arc. The first two blue arcs of Q^3 map to the third and fourth blue arcs of S , and the third blue arc maps to the fifth blue arc.

Tunable: the more dimensions, the more strings we can capture

Approximating IDL as MCFL

- **BUT** we can add more rules, making the language a more complete underapproximation of IDL

$$P^3(x_1, yx_2, x_3) \leftarrow P^3(x_1, x_2, x_3), S(y)$$

- This can be recognized in two dimensions

((([)][])([)]([[]]))

MCFL reachability

We need to solve the reachability problem for MCFLs

We can generalize the CFL-reachability algorithm:

- Define a normal form for MCFGs
- Generalize the algorithm to work with multi-dimensional predicates
- **Runs in $O(n^{d(r+1)})$**
- Prove lower bounds for rank 1
 - ◆ $O(n^{2d-\epsilon})$ for d -MCFL(1) membership (under SETH)

Experiments

Benchmark:

Scalable and precise taint analysis for Android, **Huang et al**, 2015

Graph on objects: we report the number of pairs (u,v) such that u may affect the value of v

We compare our underapproximation with our overapproximation

Taint analysis

Benchmark	Nodes	Edges	Overapp.
backflash	544	2,048	2,625
batterydoc	1,674	4,790	3,340
droidkongfu	734	1,983	3,401
fakebanker	434	1,103	251
fakedaum	1,144	2,603	2,282
faketaobao	222	450	59
jollyserv	488	998	164
loozfon	152	323	93
roidsec	553	2,026	13,052
uranai	568	1,246	143
zertsecurity	281	710	794
sciplex	1,809	5,820	TO

Taint analysis

Benchmark	Nodes	Edges	Overapp.
backflash	544	2,048	2,625
batterydoc	1,674	4,790	3,340
droidkongfu	734	1,983	3,401
fakebanker	434	1,103	251
fakedaum	1,144	2,603	2,282
faketaobao	222	450	59
jollyserv	488	998	164
loozfon	152	323	93
roidsec	553	2,026	13,052
uranai	568	1,246	143
zertsecurity	281	710	794
scipix	1,809	5,820	TO

MCFL Underapprox.

1 dim	2 dim
2,625	2,625
2,804	2,839
2,906	3,361
249	251
1,132	2,276
57	59
155	164
76	93
12,284	13,052
143	143
779	794
1,699	2,115

Taint analysis

- Optimal for **8/11** benchmarks
- We confirm **97.9%** of the taints
- Uncertain pairs reduced to less than **2.1%**

MCFL
Underapprox.

1 dim	2 dim
2,625	2,625
2,804	2,839
2,905	3,361
249	251
1,132	2,276
57	59
155	164
75	93
12,284	13,052
143	143
779	794
1,699	2,115

Taint analysis

MCFL
Underapprox.

Benchmark	Nodes	Edges	Overapp.	Time	1 dim	2 dim	Time
backflash	544	2,048	2,625	2m 51.5s	2,625	2,625	1m 34.9s
batterydoc	1,674	4,790	3,340	30m 15.5s	2,804	2,839	38.1s
droidkongfu	734	1,983	3,401	2h 18m 30.4s	2,906	3,361	28m 48s
fakebanker	434	1,103	251	12.5s	249	251	3.9s
fakedaum	1,144	2,603	2,282	3m 5.2s	1,132	2,276	50m 47.2s
faketaobao	222	450	59	8.2s	57	59	0.8s
jollyserv	488	998	164	6.2s	155	164	1.5s
loozfon	152	323	93	4.2s	76	93	3.6s
roidsec	553	2,026	13,052	3m 14s	12,284	13,052	11m 6.5s
uranai	568	1,246	143	4.1s	143	143	1.6s
zertsecurity	281	710	794	9.6s	779	794	20.3s
scipix	1,809	5,820	TO	TO	1,699	2,115	7m 16.6s

Conclusion

- Parameterization is a powerful tool that can help solve otherwise hard problems efficiently
- Structural bounds are often present in real-world graphs and can help design practical algorithms
- Lots of work to be done in finding these bounds and developing new algorithms to solve specialized problems, as was done in this thesis

Publication list

[G.K. Conrado](#), A. Pavlogiannis

CFL-based Methods for Approximating Interleaved Dyck Reachability

International Journal on Software Tools for Technology Transfer, STTT 2025

[G.K. Conrado](#), A.H. Kjelstrøm, A. Pavlogiannis, J. van de Pol

Program Analysis via Multiple Context Free Language Reachability

Principles of Programming Languages, POPL 2025

[G.K. Conrado](#), A.K. Goharshady, P. Hudec, Pingjiang Li, H.J. Motwani

Faster Treewidth-based Approximations for Wiener Index

International Symposium on Experimental Algorithms, SEA 2024

[G.K. Conrado](#), A. Pavlogiannis

A Better Approximation for Interleaved Dyck Reachability

International Workshop on the State Of the Art in Program Analysis, SOAP 2024

[G.K. Conrado](#), A.K. Goharshady, K.N. Long Nguyen

PYRAMID: A Protocol for Private and Trustless Multi-level Marketing on the Blockchain

IEEE BCCA 2024

[G.K. Conrado](#), A.K. Goharshady, C.K. Lam

The Bounded Pathwidth of Control-Flow Graphs

OOPSLA 2023

[G.K. Conrado](#), A.K. Goharshady, K. Kochev, Y.C. Tsai, A.K. Zaher

Exploiting the Sparseness of Control-flow and Call Graphs for Efficient and On-demand Algebraic Program Analysis

OOPSLA 2023

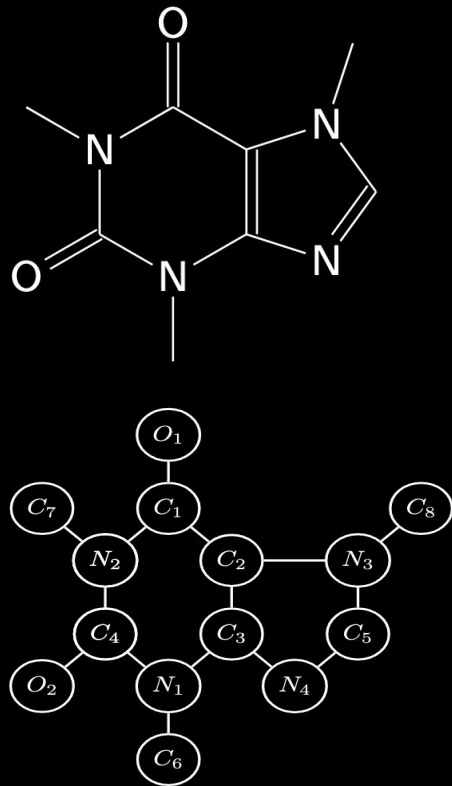
[G.K. Conrado](#), A.K. Goharshady, H.J. Motwani, S. Novozhilov

Parameterized Algorithms for Topological Indices in Chemistry

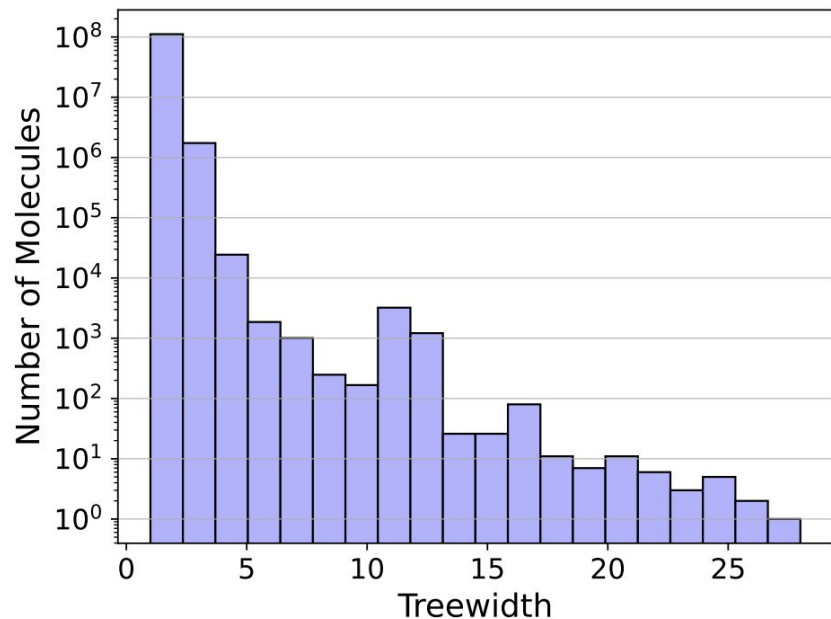
Under submission

Appendix

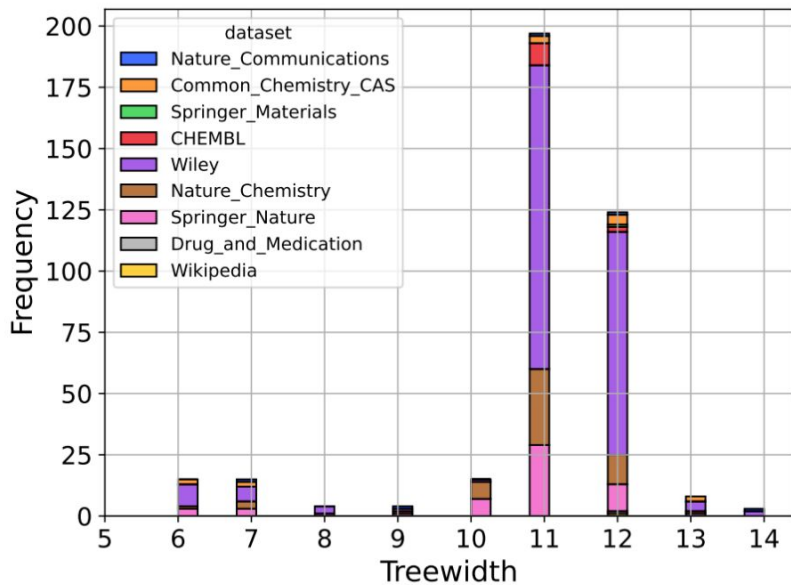
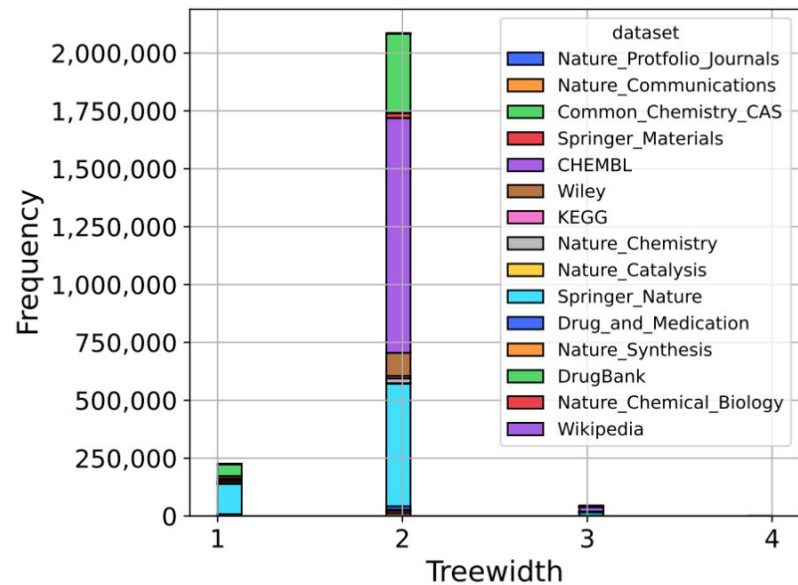
Computational Chemistry

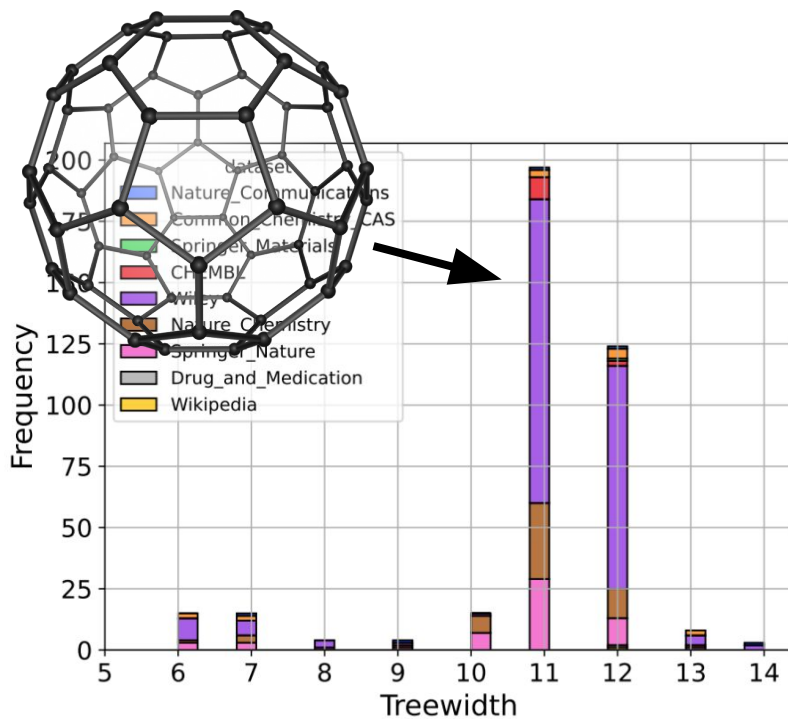
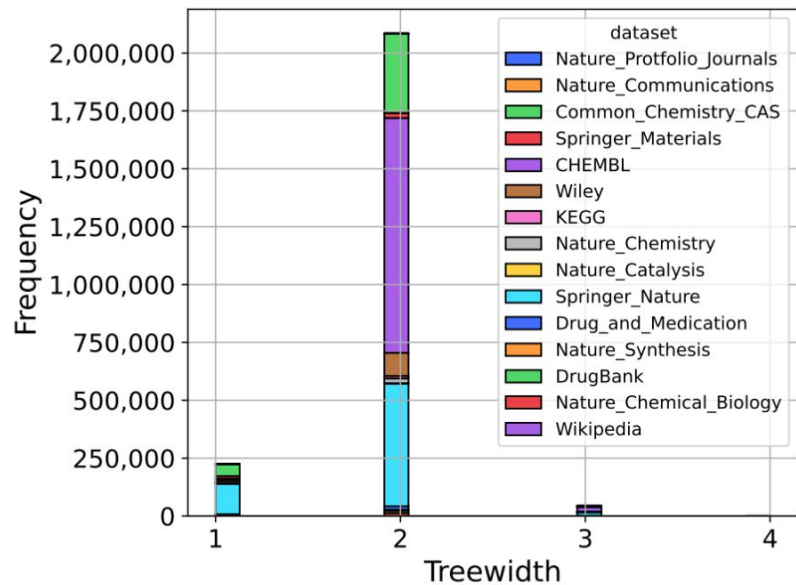


Molecules tend to have bounded treewidth



Molecules in the PubChem database





Chemical indices are used to estimate properties of molecules.
Can we speed up their computation using treewidth?

Chemical indices are used to estimate properties of molecules.
Can we speed up their computation using treewidth? **Yes!**

G.K. Conrado, A.K. Goharshady, P. Hudec, Pingjiang Li, H.J. Motwani

Faster Treewidth-based Approximations for Wiener Index

SEA 2024

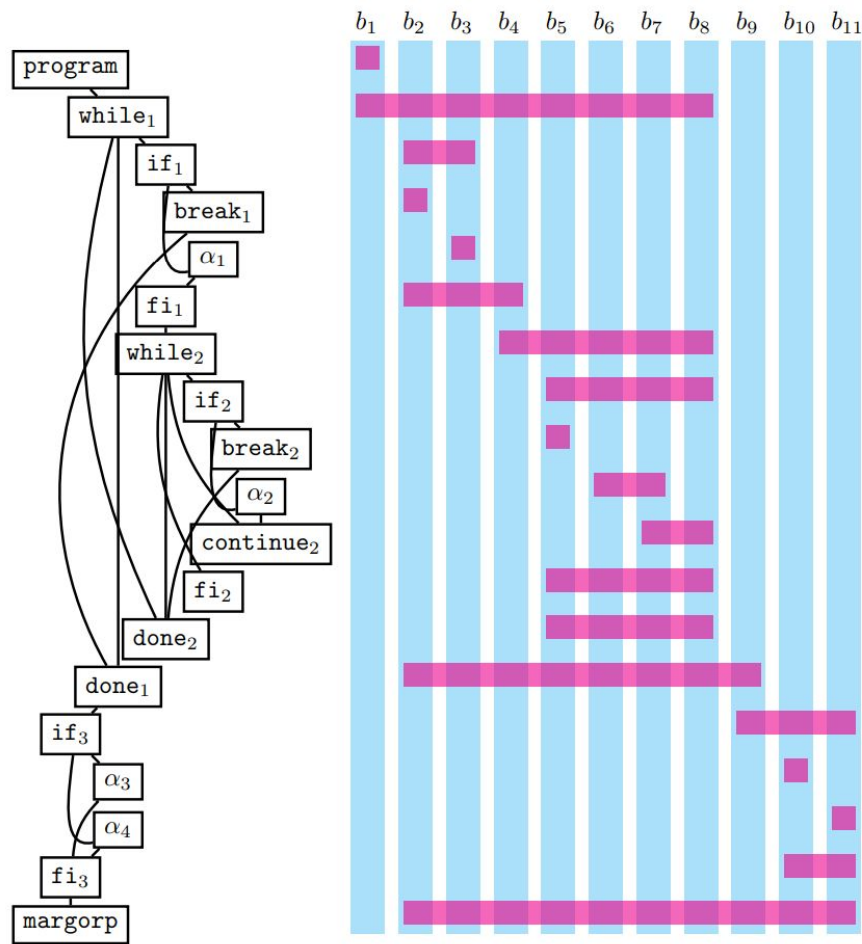
G.K. Conrado, A.K. Goharshady, H.J. Motwani, S. Novozhilov

Parameterized Algorithms for Topological Indices in Chemistry

Under submission

The Bounded Pathwidth of Control-Flow Graphs

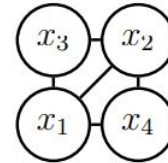
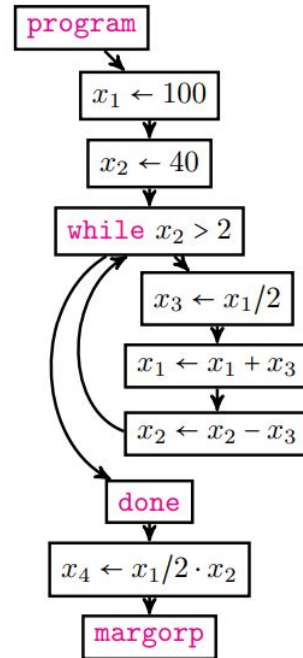
$$O(n \cdot r^{2 \cdot \text{tw} \cdot r + 2 \cdot r}) \text{ to } O(n \cdot \text{pw} \cdot r^{\text{pw} \cdot r + r + 1})$$

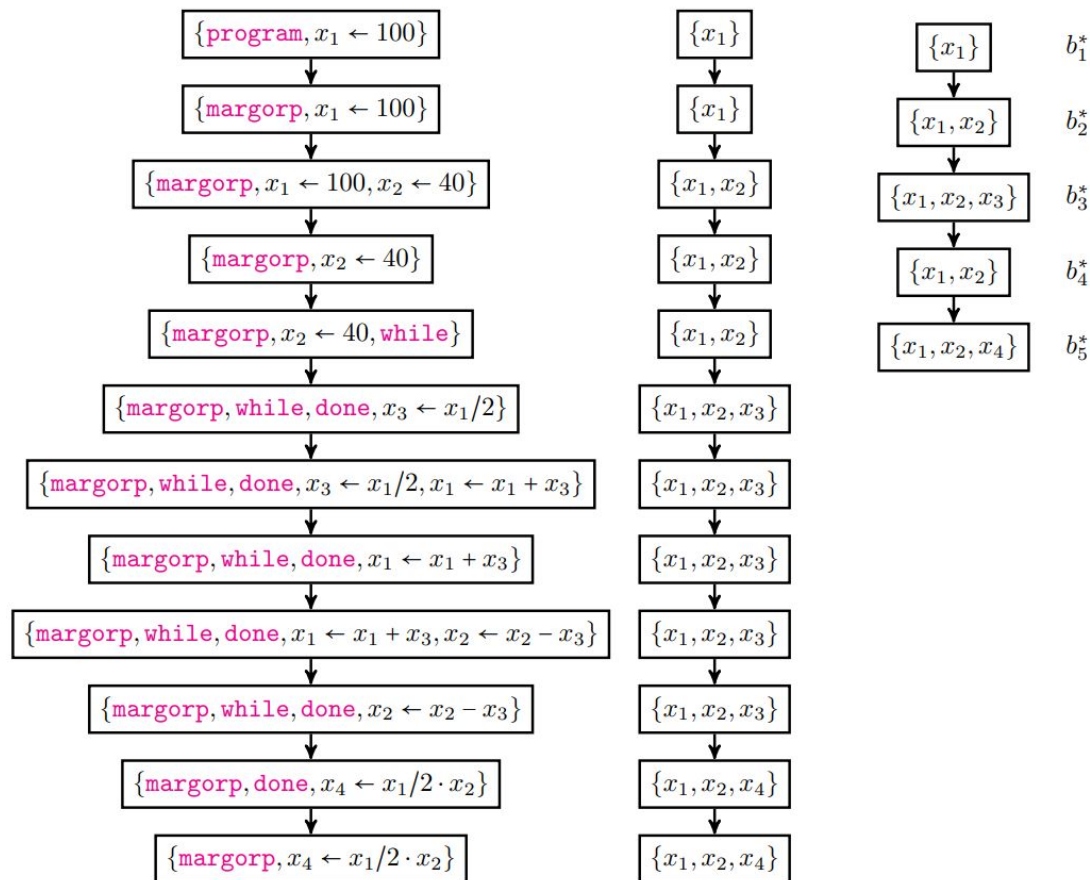


```

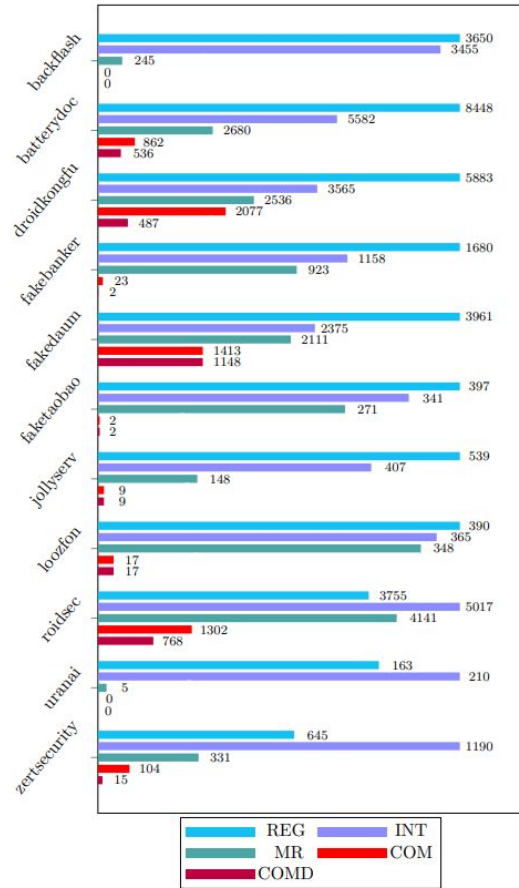
program      {}
   $x_1 \leftarrow 100;$      $\{x_1\}$ 
   $x_2 \leftarrow 40;$      $\{x_1, x_2\}$ 
  while  $x_2 > 2$  do  $\{x_1, x_2\}$ 
     $x_3 \leftarrow x_1/2$   $\{x_1, x_2, x_3\}$ 
     $x_1 \leftarrow x_1 + x_3$   $\{x_1, x_2, x_3\}$ 
     $x_2 \leftarrow x_2 - x_3$   $\{x_1, x_2, x_3\}$ 
  done;       $\{x_1, x_2\}$ 
   $x_4 \leftarrow x_1/2 \cdot x_2$   $\{x_1, x_2, x_4\}$ 
margorp    {}

```





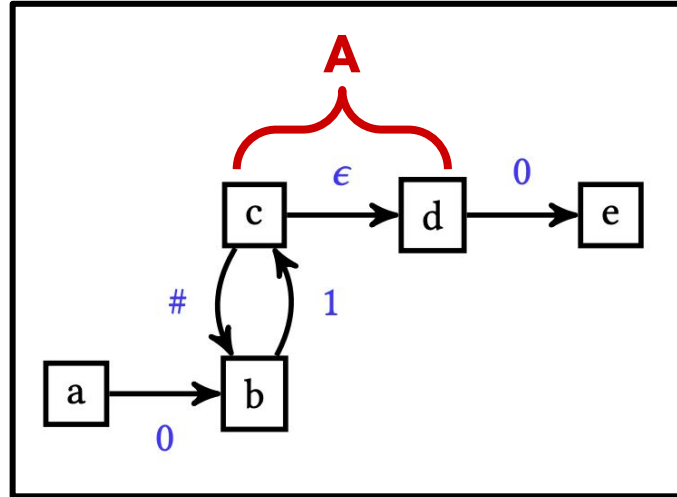
CFL-based Methods for Approximating Interleaved Dyck Reachability



Program Analysis via Multiple Context Free Language Reachability

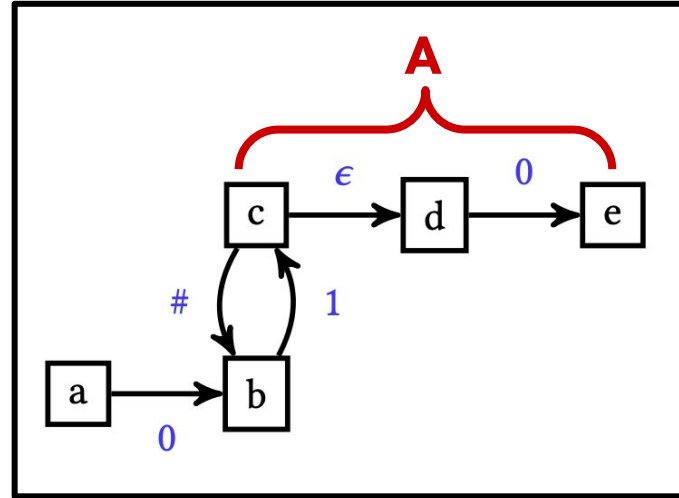
CFL reachability

$A(\epsilon)$

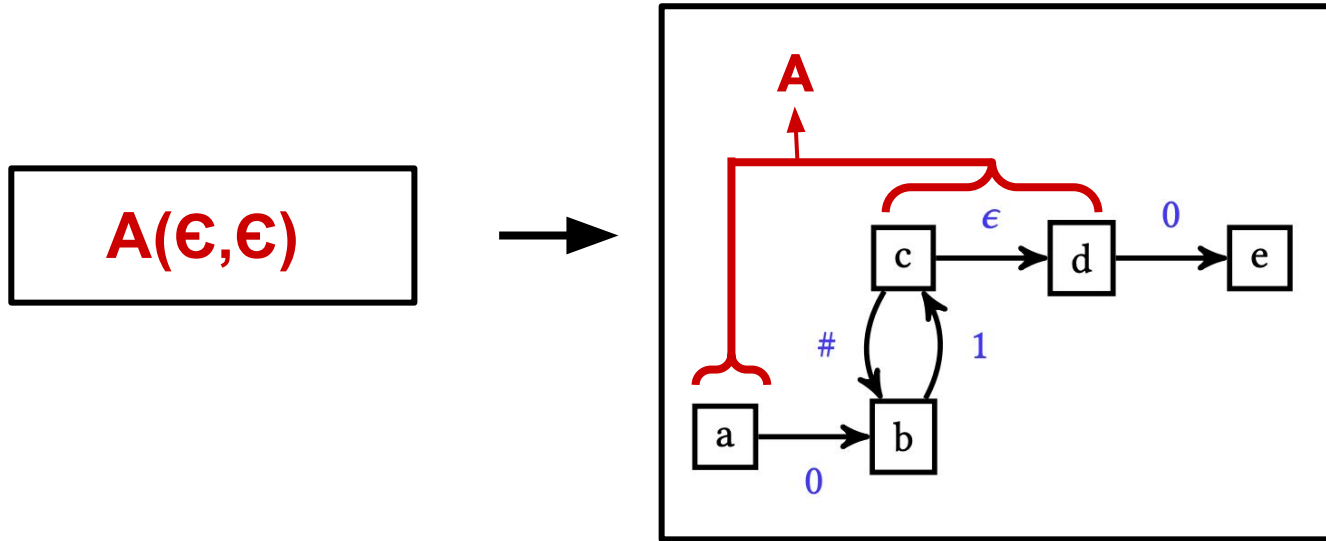


MCFL reachability

$A(x0) \leftarrow A(x)$

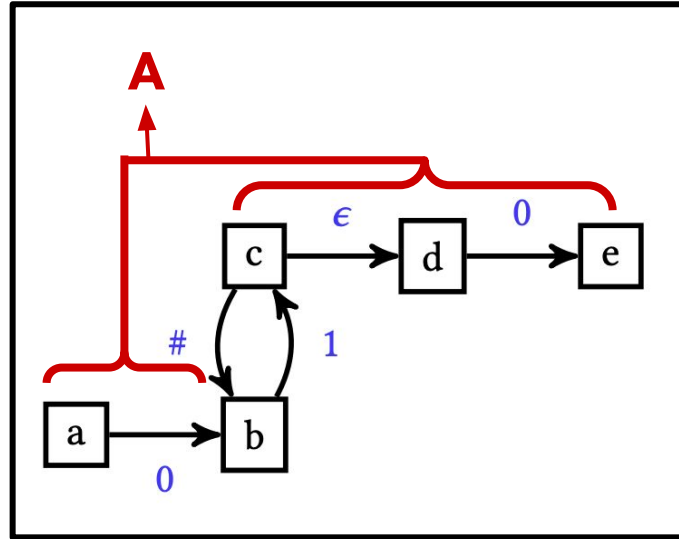


MCFL reachability



MCFL reachability

$A(x0, x0) \leftarrow A(x, x)$



Comparison with MPDS approximations

- Stronger than bounded context-switchings for same number of contexts
- Incomparable to bounded phases
 - ◆ MCFL of dimension $\mathbf{d}$ recognizes strings not recognized by $\mathbf{k}$ phases for any $\mathbf{k}$
 - ◆ $\mathbf{k}$ phases recognizes strings not recognized by an MCFL of dimension $\mathbf{d}$ for any $\mathbf{d}$

Comparison with MPDS approximations

- MCFL reachability has a single exponential dependency on the number of dimensions **d**
- Reachability for bounded phase multi-stack pushdown automata has a double exponential dependency on the number of phases **k**